Entraîner un modèle simple à partir de journaux d’infrastructure

Détection SSH chez Hexalp

Preuve de travail destinée au formateur Olivier HIMBLOT • 21 octobre 2026

Critère évalué

Un modèle Scikit-learn est entraîné de façon reproductible sur des journaux préparés et des caractéristiques justifiées, via un script commenté.

Preuves présentées

Script detection_ssh.py et requirements.txt ; CSV d’alertes ; fiche du modèle ; résultats des notebooks observés en captures. Le code complet du script figure en annexe.

Périmètre établi

Données fictives du bastion Hexalp. Captures de travail de l’apprenant dans Jupyter et exécution du script dans sa VM à graine 7. Vérification séparée par l’assistant à graines 42 et 7, dont deux exécutions identiques à 42. Le script a été rédigé avec assistance : commentaires à relire et expliquer personnellement.

Pièces à compléter avant remise

00_python_survie.ipynb : fichier local contrôlé, 0/23 cellules de code avec compteur d’exécution. Les captures attestent des résultats, mais ne remplacent pas le notebook complet sauvegardé.

01_detection_ssh.ipynb : fichier local contrôlé, 0/27 cellules de code avec compteur d’exécution. Les captures attestent des résultats, mais ne remplacent pas le notebook complet sauvegardé.

Les deux notebooks complétés et exécutés de la VM sont à rapatrier. Aucune exécution de tous les notebooks n’est revendiquée à partir des seuls originaux. Aucun déploiement hebdomadaire ni blocage automatique n’est réalisé.

1. Critère, livrables et préparation

Attendu du critère	Travail fourni et portée de la preuve
Journaux préparés	CSV fictif inspecté ; valeurs manquantes localisées ; retrait documenté, sans assimiler absence de mesure à zéro.
Caractéristiques justifiées	Sept variables de comportement ; étiquette exclue de X, contrôle fail2ban réservé à la comparaison ; date/IP conservées pour l'analyste.
Modèle entraîné	Captures fit et matrice du notebook ; script complet relançable. Arbre profondeur 3 et Isolation Forest sans y.
Reproductibilité	Deux exécutions à 42 identiques dans l'environnement de vérification ; essai distinct à 7. Versions et empreintes conservées.
Preuve de travail	Script, requirements, CSV de 111 alertes et fiche modèle disponibles. Notebooks exécutés complets de la VM à récupérer.

Données et nettoyage

Source : hexalp_ssh_sept2026.csv, 14–27 septembre 2026. Une ligne résume une IP pendant une heure. Brut : 1 704 observations, 11 colonnes, 117 attaques (6,87 %). Les étiquettes sont attribuées à Hugo dans le scénario.

90 volumes sont absents pendant deux pannes NetFlow : 19 septembre, tranches 0–11 h (20 lignes), et 22 septembre, 10–15 h (70 lignes). Retrait de ces 90 lignes, dont 9 attaques ; restent 1 614 observations et 108 attaques (6,69 %). Ces lignes restent hors couverture : ce choix ne résout pas la panne de collecte.

Variables d'entrée et exclusions

Retenues : heure, externe (origine validée puis codée 0/1), echecs_auth, succes_auth, comptes_distincts, duree_session_min, octets_sortants_mo. Elles décrivent l'activité de la tranche terminée ; elles ne permettent pas de prétendre détecter avant la fin de cette tranche.

Exclues : etiquette, réponse réservée à l'apprentissage supervisé/évaluation ; fail2ban_ban, pour éviter de recopier un contrôle qu'on compare ; ip_source et date, pour limiter la mémorisation dans ce premier modèle. La date et l'IP restent dans les alertes pour les retrouver.

2. Entraînement et résultats

Séparation stratifiée à graine 42 : 1 129 lignes d'entraînement (76 attaques) et 485 de test (32 attaques). Une attaque de plus ou de moins modifie le rappel de 3,125 points sur ce test.

Périmètre	FP	FN	VP	Précision	Rappel
Arbre, test 485	0	4	28	100 %	87,50 %
fail2ban, même test	0	15	17	100 %	53,125 %
Isolation Forest, ajustement 1 614	5	64	44	89,80 %	40,74 %
Union, descriptive sur tout X	5	2	106	95,50 %	98,15 %

Les deux dernières lignes utilisent le jeu d'ajustement d'Isolation Forest ; l'union inclut les données d'entraînement de l'arbre. Elles ne constituent pas un résultat de généralisation comparable directement au test supervisé.

Lecture de l'arbre : explicable mais fragile

Profondeur 3, six feuilles. Avec au plus trois comptes : durée ≤0,5 minute ou >64 minutes → attaque ; sinon normal. À partir de quatre comptes : externe → attaque ; interne → attaque jusqu'à 19 h, normal dès 20 h. Le seuil horaire interne est un raccourci appris, pas une autorisation métier.

Les quatre attaques manquées ont un seul compte et des sessions de 24 à 57 minutes. Cet arbre ne teste directement ni le volume sortant ni les échecs, malgré leur présence dans les variables proposées.

Anomalies : classement utile, coût de détection

À contamination 0,03 : 49 signalées, dont 44 attaques de référence et 5 normales — scanner 3, sauvegarde 2. Le top 15 est entièrement étiqueté attaque ; les qualifications analyste écrites avant révélation ne sont pas montrées. Les trois lignes SSH de Julien pendant l'incident WMS ne sont pas signalées ; le CSV ne contient pas les données envoyées à son chatbot.

Contamination	Alertes / 2 sem.	VP	FP	FN
0,03	49	44	5	64
0,05	81	64	17	44
0,10	162	104	58	4

Choisir après comparaison sur le même jeu fragilise une évaluation finale : prévoir un nouveau test temporel et suivre les pratiques qui changent.

3. Script reproductible et charge réelle

Chaîne du script

Paramètres regroupés ; chemins relatifs au fichier du script ; contrôles de schéma et de valeurs ; nettoyage ; construction X/y ; partage ; entraînement et évaluation ; export des lignes anomalie OU avis_arbre=1. Le CSV est trié de manière déterministe. Le code ne réalise ni appel réseau, ni ban, ni modification de compte.

Essais effectués par l'assistant, distincts de la VM

Venv temporaire Linux/x86_64, Python 3.14.4, pandas 3.0.6, scikit-learn 1.9.1, matplotlib 3.11.2, JupyterLab 4.6.4, NumPy 2.5.4 et SciPy 1.18.1. Les versions directes du kit sont respectées ; pip check : aucune incompatibilité signalée. Les dépendances transitives sont relevées, pas protégées par un fichier de hachages approuvé.

Deux exécutions à graine 42 : console et CSV identiques octet par octet. Essai à graine 7 dans un autre fichier : règles différentes malgré 0 FP et 4 FN également pour l'arbre sur son nouveau test ; IF : 4 FP/63 FN/45 VP. Les exports ont 107 observations communes, 4 propres à 42 et 1 propre à 7.

Empreinte du CSV final à graine 42

3bb985d4cfb64d31c204404d4300f7c8cb8eb47b2c0979ed5c28d4a438a24b40

L'union dépasse le budget de 30 alertes/semaine

Semaine	IF seules, graine42	Union42 exportée	Union7
14–20 septembre	20	45	44
21–27 septembre	29	66	64

Le CSV final contient 111 lignes, plus l'en-tête. La file union moyenne est 55,5/semaine, pas 24,5. Diminuer la contamination ne supprime pas les alertes de l'arbre seul. Aucun plafond caché n'a été appliqué. Une file IF seule respecterait ici la capacité, mais manquerait 64 attaques : arbitrage RSSI requis, pas acceptation automatique.

Avant exploitation : évaluer regroupement, priorités et détection sur données indépendantes ; séparer entraînement validé et prédiction sans étiquettes ; versionner le modèle ; superviser qualité de collecte, dérive, tâche planifiée et file analyste. Aucun cron n'a été créé.

4. Fiche du modèle — Prototype J3

Rubrique	Fiche synthétique — 21 octobre 2026
Données	Bastion Hexalp fictif, 14–27/09/2026 ; 1 704 lignes, 117 attaques (6,87 %). Après nettoyage : 1 614 lignes, 108 attaques (6,69 %).
Préparation	90 volumes manquants retirés, dont neuf attaques ; hors couverture explicite. Origine vérifiée puis codée interne0/externe1. Aucun volume inconnu remplacé par zéro.
Caractéristiques	Heure, externe, échecs, succès, comptes, durée et volume. Exclusions : étiquette (réponse), fail2ban (indépendance), date/IP (mémorisation). Identifiants conservés pour l'analyste.
Modèles et réglages	DecisionTreeClassifier max_depth=3, test stratifié30 % ; IsolationForest contamination=0,03, sans y, sur tout X ; graine42. Export union.
Résultats	Arbre test : 0FP/4FN, précision100 %, rappel87,5 %. IF ajustement : 5FP/64FN, précision89,80 %, rappel40,74 %. Union descriptive : 5FP/2FN ; 111 alertes, 45 puis66/semaine ; dépassement du budget.
Limites	90 lignes hors couverture ; pas de contenu web dans les agrégats SSH ; branche interne fragile ; audits/sauvegardes atypiques ; test réutilisé pour réglages ; IF/union non évalués sur données nouvelles ; dérive possible.
Conditions d'utilisation	Analyste SOC/astreinte qualifie ; Nadia arbitre charge et risques. Aucun blocage. Réévaluer avant production et après changement des données, règles ou usages ; conserver modèle, paramètres, versions, preuves et retour arrière.

Statut de remise des fichiers

Disponibles dans ia01-j3 : detection_ssh.py, requirements.txt, alertes_a_qualifier.csv, carnet_J3.md et preuves de vérification. À joindre après récupération : les deux notebooks complets sauvegardés depuis la VM. L'évaluation finale appartient au formateur ; ce document ne la présume pas.

5. Preuve visuelle — Notebook supervisé

Partie C — Le classifieur supervisé

On met de côté 30 % des observations pour l'évaluation. Le modèle ne les verra **jamais** pendant l'entraînement : c'est votre jeu de test du J1.

`stratify=y` garantit la même proportion d'attaques dans les deux jeux.

[17]:
```
df = df.dropna(subset=["octets_sortants_mo"]).copy()

X = df[CARACTERISTIQUES]
y = (df["etiquette"] == "attaque").astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    stratify=y,
    random_state=GRAINE,
)

print("Entraînement :", X_train.shape, "Test :", X_test.shape)
```

```
Entraînement : (1129, 7) Test : (485, 7)
```

Un **arbre de décision** fait ce que vous avez fait au J1 : il cherche la caractéristique et le seuil qui séparent le mieux les exemples, puis recommence dans chaque branche. `max_depth` limite le nombre de questions successives.

[19]:
```
arbre = DecisionTreeClassifier(max_depth=3, random_state=GRAINE)
arbre.fit(X_train, y_train)

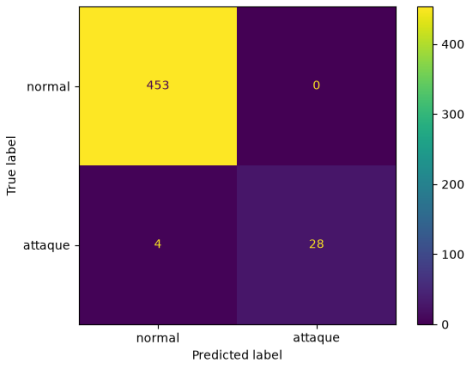
predictions = arbre.predict(X_test)
print("Accuracy :", round(accuracy_score(y_test, predictions), 3))
```

```
Accuracy : 0.992
```

Capture apprenant 10:48:58 : nettoyage, partage 1 129/485, entraînement profondeur3 et accuracy0,992.

✍ **C1.** Traduisez les règles de l'arbre en français, comme vous l'aviez fait au J1. Retrouvez-vous votre règle ? Une branche vous paraît-elle dangereuse ? *(Regardez la branche des connexions internes.)*

[21]:
```
ConfusionMatrixDisplay.from_predictions(y_test, predictions, display_labels=["normal", "attaque"])
plt.show()
print(classification_report(y_test, predictions, target_names=["normal", "attaque"]))
```



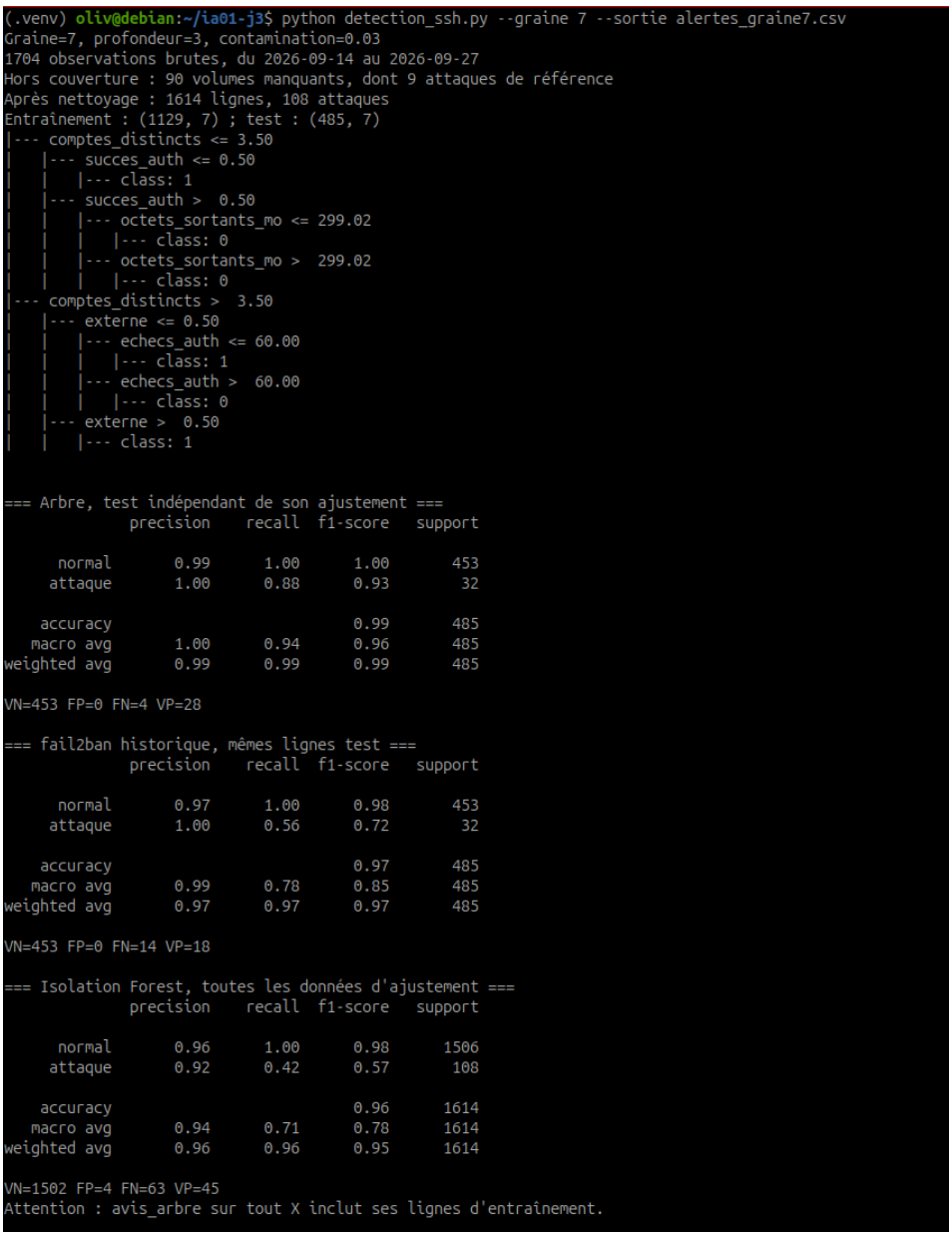

```
              precision    recall  f1-score   support

      normal       0.99      1.00      1.00       453
     attaque       1.00      0.88      0.93        32

    accuracy                           0.99       485
   macro avg       1.00      0.94      0.96       485
weighted avg       0.99      0.99      0.99       485
```

Capture apprenant 10:49:21 : matrice 453VN/0FP/4FN/28VP et rapport sur 32 attaques. Les résultats ne prouvent ni exécution complète ni sauvegarde du notebook dans les fichiers remis.

6. Preuve visuelle — Exécution dans la VM



```
(.venv) oliv@debian:~/ia01-j3$ python detection_ssh.py --graine 7 --sortie alertes_graine7.csv
Graine=7, profondeur=3, contamination=0.03
1704 observations brutes, du 2026-09-14 au 2026-09-27
Hors couverture : 90 volumes manquants, dont 9 attaques de référence
Après nettoyage : 1614 lignes, 108 attaques
Entraînement : (1129, 7) ; test : (485, 7)
|--- comptes_distincts <= 3.50
|   |--- succes_auth <= 0.50
|   |   |--- class: 1
|   |--- succes_auth >  0.50
|   |   |--- octets_sortants_mo <= 299.02
|   |   |   |--- class: 0
|   |   |--- octets_sortants_mo >  299.02
|   |   |   |--- class: 0
|--- comptes_distincts >  3.50
|   |--- externe <= 0.50
|   |   |--- echecs_auth <= 60.00
|   |   |   |--- class: 1
|   |   |--- echecs_auth >  60.00
|   |   |   |--- class: 0
|   |--- externe >  0.50
|   |   |--- class: 1


=== Arbre, test indépendant de son ajustement ===
              precision    recall  f1-score   support

      normal       0.99      1.00      1.00       453
     attaque       1.00      0.88      0.93        32

    accuracy                           0.99       485
   macro avg       1.00      0.94      0.96       485
weighted avg       0.99      0.99      0.99       485

VN=453 FP=0 FN=4 VP=28

=== fail2ban historique, mêmes lignes test ===
              precision    recall  f1-score   support

      normal       0.97      1.00      0.98       453
     attaque       1.00      0.56      0.72        32

    accuracy                           0.97       485
   macro avg       0.99      0.78      0.85       485
weighted avg       0.97      0.97      0.97       485

VN=453 FP=0 FN=14 VP=18

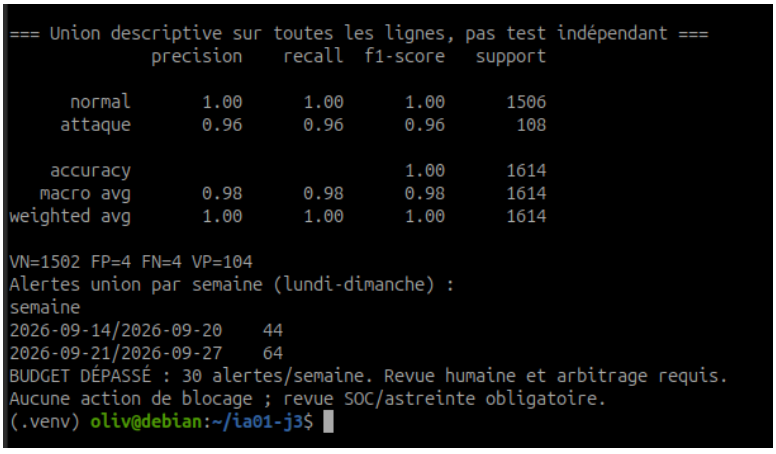
=== Isolation Forest, toutes les données d'ajustement ===
              precision    recall  f1-score   support

      normal       0.96      1.00      0.98      1506
     attaque       0.92      0.42      0.57       108

    accuracy                           0.96      1614
   macro avg       0.94      0.71      0.78      1614
weighted avg       0.96      0.96      0.95      1614

VN=1502 FP=4 FN=63 VP=45
Attention : avis_arbre sur tout X inclut ses lignes d'entrainement.
```

Capture 11:20:40 : commande à graine7 dans le venv Debian ; préparation, règles et rapports. Résultats visibles concordants avec l'essai séparé de l'assistant.



```
=== Union descriptive sur toutes les lignes, pas test indépendant ===
              precision    recall  f1-score   support

      normal       1.00      1.00      1.00      1506
     attaque       0.96      0.96      0.96       108

    accuracy                           1.00      1614
   macro avg       0.98      0.98      0.98      1614
weighted avg       1.00      1.00      1.00      1614

VN=1502 FP=4 FN=4 VP=104
Alertes union par semaine (lundi-dimanche) :
semaine
2026-09-14/2026-09-20    44
2026-09-21/2026-09-27    64
BUDGET DÉPASSÉ : 30 alertes/semaine. Revue humaine et arbitrage requis.
Aucune action de blocage ; revue SOC/astreinte obligatoire.
(.venv) oliv@debian:~/ia01-j3$
```

Capture 11:20:55 : 44 puis64 alertes/semaine, avertissement budget et retour au prompt. Pas de preuve par ces captures de deux relances42 dans la VM ou d'identité des CSV entre machines.

Annexe — Script commenté complet

Version fournie dans ia01-j3/detection_ssh.py. Code rédigé avec assistance ; l'apprenant doit pouvoir expliquer les choix. Squelette d'origine conservé. Les seuls effets sont lecture du CSV, calculs, affichage et écriture du CSV demandé.

```
001 """Évaluer deux modèles SSH sur le kit fictif Hexalp et exporter des alertes.
002
003 IA-01 / J3 — Olivier HIMBLOT, avec assistance à la rédaction, 21 octobre 2026.
004 Usage : python detection_ssh.py [--graine 42] [--entree fichier.csv] [--sortie alertes.csv]
005 Ce démonstrateur réentraîne les modèles et exige des étiquettes : il n'est pas
006 un service de détection hebdomadaire prêt à la production. Aucune action sur le SI.
007 Les commentaires sont une proposition à relire et expliquer par l'apprenant.
008 """
009
010 import argparse
011 from pathlib import Path
012
013 import numpy as np
014 import pandas as pd
015 from sklearn.ensemble import IsolationForest
016 from sklearn.metrics import classification_report, confusion_matrix
017 from sklearn.model_selection import train_test_split
018 from sklearn.tree import DecisionTreeClassifier, export_text
019
020 # Les chemins par défaut partent du script, pas du terminal de lancement.
021 DOSSIER = Path(__file__).resolve().parent
022 FICHIER_ENTREE = DOSSIER / "hexalp_ssh_sept2026.csv"
023 FICHIER_ALERTES = DOSSIER / "alertes_a_qualifier.csv"
024 GRAINE = 42
025
026 # On décrit le comportement, sans donner la réponse ni copier le contrôle fail2ban.
027 # Date et IP restent dans le résultat pour que l'analyste retrouve les événements.
028 CARACTERISTIQUES = [
029     "heure", "externe", "echecs_auth", "succes_auth", "comptes_distincts",
030     "duree_session_min", "octets_sortants_mo",
031 ]
032 PROFONDEUR_ARBRE = 3
033 CONTAMINATION = 0.03
034 # Garder les mêmes réglages que les captures facilite la comparaison.
035 # L'export retient l'union : une attaque vue par un seul modèle reste examinable.
036 # Ce choix peut dépasser le budget d'astreinte ; il est mesuré, pas caché.
037
038
039 def bilan(nom, reference, prediction):
040     """Afficher des comptes explicites, en conservant l'ordre normal / attaque."""
041     tn, fp, fn, tp = confusion_matrix(reference, prediction, labels=[0, 1]).ravel()
042     print(f"\n=== {nom} ===")
043     print(classification_report(
044         reference, prediction, labels=[0, 1],
045         target_names=["normal", "attaque"], zero_division=0,
046     ))
047     print(f"VN={tn} FP={fp} FN={fn} VP={tp}")
048
049
050 def main():
051     parser = argparse.ArgumentParser(description=__doc__)
052     parser.add_argument("--entree", type=Path, default=FICHIER_ENTREE)
053     parser.add_argument("--sortie", type=Path, default=FICHIER_ALERTES)
054     parser.add_argument("--graine", type=int, default=GRAINE)
055     args = parser.parse_args()
056     if args.entree.resolve() == args.sortie.resolve():
057         parser.error("La sortie doit être distincte du CSV d'entrée.")
058
059     # Refuser un schéma inattendu évite de produire silencieusement de faux avis.
060     df = pd.read_csv(args.entree)
061     attendues = {
062         "date", "heure", "ip_source", "origine", "echecs_auth", "succes_auth",
063         "comptes_distincts", "duree_session_min", "octets_sortants_mo",
064         "fail2ban_ban", "etiquette",
065     }
066     if not attendues.issubset(df.columns):
067         raise ValueError(f"Colonnes manquantes : {sorted(attendues - set(df.columns))}")
068     if df.empty:
069         raise ValueError("Le fichier ne contient aucune observation.")
070     if df[list(attendues - {"octets_sortants_mo"})].isna().any().any():
071         raise ValueError("Champ obligatoire manquant hors volume NetFlow.")
072     if not df["origine"].isin(["interne", "externe"]).all():
073         raise ValueError("Origine inconnue : ne pas la convertir en interne.")
074     if not df["etiquette"].isin(["normal", "attaque"]).all():
075         raise ValueError("Étiquette inconnue : jeu évalué obligatoirement qualifié.")
```

```
076
077     # Valider les mesures avant toute suppression : une chaîne illisible n'est pas zéro.
078     mesures = [c for c in CARACTERISTIQUES if c != "externe"]
079     df[mesures] = df[mesures].apply(pd.to_numeric, errors="raise")
080     valeurs = df[mesures].to_numpy(dtype=float)
081     if np.isinf(valeurs).any() or (df[mesures] < 0).any().any():
082         raise ValueError("Mesure négative ou infinie.")
083     if not df["heure"].between(0, 23).all() or (df["heure"] % 1 != 0).any():
084         raise ValueError("L'heure doit être un entier de 0 à 23.")
085     dates_brutes = pd.to_datetime(df["date"], format="%Y-%m-%d", errors="raise")
086     print(f"Graine={args.graine}, profondeur={PROFONDEUR_ARBRE}, contamination={CONTAMINATION}")
087     print(f"{len(df)} observations brutes, du {dates_brutes.min().date()} au
    {dates_brutes.max().date()}")
088
089     # Une sonde en panne n'a pas mesuré zéro octet. On reproduit le choix du TP :
090     # ces lignes sont exclues du modèle complet et explicitement comptées hors couverture.
091     absentes = df["octets_sortants_mo"].isna()
092     print(f"Hors couverture : {absentes.sum()} volumes manquants, "
093           f"dont {(df.loc[absentes, 'etiquette'] == 'attaque').sum()} attaques de référence")
094     df = df.loc[~absentes].copy()
095     df["externe"] = (df["origine"] == "externe").astype(int)
096     X = df[CARACTERISTIQUES]
097     y = (df["etiquette"] == "attaque").astype(int)
098     if y.nunique() != 2 or y.value_counts().min() < 2:
099         raise ValueError("Deux classes avec assez d'exemples sont nécessaires au partage stratifié.")
100
101     # Réserver le même test que le notebook : ni l'IP, ni l'étiquette, ni fail2ban
102     # n'entrent dans X. Le partage aléatoire reste une limite pour des séries temporelles.
103     X_train, X_test, y_train, y_test = train_test_split(
104         X, y, test_size=0.3, stratify=y, random_state=args.graine,
105     )
106     print(f"Après nettoyage : {len(df)} lignes, {int(y.sum())} attaques")
107     print(f"Entraînement : {X_train.shape} ; test : {X_test.shape}")
108     arbre = DecisionTreeClassifier(max_depth=PROFONDEUR_ARBRE, random_state=args.graine)
109     arbre.fit(X_train, y_train)
110     predictions = arbre.predict(X_test)
111     print(export_text(arbre, feature_names=CARACTERISTIQUES))
112     bilan("Arbre, test indépendant de son ajustement", y_test, predictions)
113     bilan("fail2ban historique, mêmes lignes test", y_test, df.loc[X_test.index, "fail2ban_ban"])
114
115     # Comme dans la partie D, l'Isolation Forest voit toutes les observations sans y.
116     # Comparer ensuite à y aide l'analyse, mais ce n'est PAS un test sur données nouvelles.
117     iforest = IsolationForest(contamination=CONTAMINATION, random_state=args.graine)
118     iforest.fit(X)
119     df["score_anomalie"] = iforest.decision_function(X)
120     df["anomalie"] = iforest.predict(X) == -1
121     df["avis_arbre"] = arbre.predict(X)
122     bilan("Isolation Forest, toutes les données d'ajustement", y, df["anomalie"].astype(int))
123     print("Attention : avis_arbre sur tout X inclut ses lignes d'entraînement.")
124
125     # L'union préserve les signaux complémentaires. Le tri stable et ses départages
126     # rendent l'export déterministe à données et environnement identiques.
127     masque = df["anomalie"] | (df["avis_arbre"] == 1)
128     alertes = df.loc[masque].sort_values(
129         ["score_anomalie", "date", "heure", "ip_source"], kind="stable",
130     )
131     colonnes = [
132         "date", "heure", "ip_source", "origine", "echecs_auth", "succes_auth",
133         "comptes_distincts", "octets_sortants_mo", "score_anomalie", "anomalie", "avis_arbre",
134     ]
135     alertes[colonnes].to_csv(args.sortie, index=False)
136     print(f"\n{len(alertes)} alertes écrites dans {args.sortie.name}")
137     bilan("Union descriptive sur toutes les lignes, pas test indépendant", y, masque.astype(int))
138
139     # Compter par semaine réelle rend visibles les pics masqués par une moyenne.
140     # Les semaines sans alerte restent affichées si des observations existent.
141     semaines = pd.to_datetime(df["date"]).dt.to_period("W-SUN")
142     hebdo = pd.DataFrame({"semaine": semaines.astype(str), "alerte": masque})
143     print("Alertes union par semaine (lundi-dimanche) :")
144     print(hebdo.groupby("semaine")["alerte"].sum().to_string())
145     if (hebdo.groupby("semaine")["alerte"].sum() > 30).any():
146         print("BUDGET DÉPASSÉ : 30 alertes/semaine. Revue humaine et arbitrage requis.")
147     print("Aucune action de blocage ; revue SOC/astreinte obligatoire.")
148
149
150 if __name__ == "__main__":
151     main()
```

Références et pièces de preuve

Pièces du dossier : kit fictif fourni, captures du 21 octobre et essais séparés de l'assistant. Référence technique : Scikit-learn, guide utilisateur, documentation consultée le 21 octobre 2026 : https://scikit-learn.org/stable/user_guide.html

Répertoire de preuves : ia01-j3/preuves/ — verification_script.md, execution_graine42.txt, execution_graine7.txt, alertes_graine7.csv, dependances_verification.txt. Les lignes de sortie et hachages servent au rapprochement ; pas d'entraînement supplémentaire pendant la génération du PDF.

Fichier	SHA-256

detection_ssh.py

90b8f9f9ded91056983e61c82b7594191ffbb0dd01a6cafbe00a9568f53a20a2

requirements.txt

1aeb643d57b014b2fb693179d02ad0dc5fada6306958d62379d99e9a10739d34

hexalp_ssh_sept2026.csv

8f9565ee5b5b9e1d034cd6171d9e68ac3a7307af0f64b3ccda096e0badb1c126

alertes_a_qualifier.csv

3bb985d4cfb64d31c204404d4300f7c8cb8eb47b2c0979ed5c28d4a438a24b40

Les empreintes correspondent aux fichiers locaux au moment de la génération. Des sorties réexécutées ou des notebooks remplacés doivent être conservés avec leur propre version et une date de vérification. Les deux notebooks restent à récupérer ; cette étape de remise n'est pas attestée par les captures seules.